C ARCH PROJECT

Implementation Report

Web simulator, memory explorer, and instruction encoding/decoding progress

Project	Report date	Status
Integrated Web-Based RISC-V Simulator and Computer Architecture Design Platform	09 November 2026	Core learning tools implemented

1. Project summary

The project is being developed as a browser-based computer architecture learning platform. The implemented foundation combines an interactive RISC-V web simulator, a memory explorer, and instruction encoding/decoding capabilities. The web interface is connected to the existing Kites C++ processor and assembler so program behavior and machine state come from the simulator engine.

The current work establishes the core execution and inspection workflow. The broader scope document also includes compiler comparison, custom ISA design, processor design, cache analysis, branch prediction, and integration/verification. Those areas remain staged for later development.

2. Design approach

React + TypeScript **Browser interface**	**HTTP / JSON** **Local API boundary**	**C++ Kites engine** **Assembler + processor state**

The browser sends assembly, run, step, reset, register, and memory requests to the local HTTP API. The API delegates assembly and execution to Kites rather than reimplementing the processor in JavaScript. It returns machine state and instruction information for the UI to display. Memory is fetched in bounded pages, allowing inspection of a sparse address space without allocating the full memory range.

Execution is driven through explicit API calls. The editor can assemble, run step by step, or run continuously with a selectable speed. The execution flow displays instruction addresses, machine encodings, source lines, and execution counters. Breakpoints can be toggled from source lines or instruction rows. Reaching the last instruction marks the program complete; registers and memory remain available until a new assembly or reset.

3. Tools and frameworks

Layer	Tools	Role in the project
Simulator and API	C++20, Qt, CMake	Reuse the Kites assembler, processor variants, register state, and memory controller; expose them through a local HTTP API.
Web client	React, TypeScript, Vite	Provide the editor, execution flow, machine-state views, and memory explorer.
Integration	HTTP + JSON	Connect browser actions to the simulator engine and return structured state and instruction data.

Implementation status

The status below reflects the current connected web project and the features identified as achieved by the project team.

4. Implemented components

Component	Implementation	Status
Web simulator	Assembly editor with assemble, run, step, and reset actions; execution flow with instruction address, machine encoding, source line, PC/cycle/retired counters, and supported Kites processor selections. Execution completion is handled at the program end.	**Implemented**
Memory explorer	Address-based memory view with segment navigation, paged reads, byte search, and editable byte values. Register and memory state can be inspected after execution.	**Implemented**
Instruction encoder and decoder	The assembler/code-generation path converts supported assembly instructions into machine encodings. Instruction information is exposed in the execution flow for inspection, alongside readable instruction forms and source mappings.	**Implemented for supported instruction set**
Breakpoints and execution state	Breakpoints can be set from the source editor or instruction flow. Execution pauses when a marked source line is reached. Completed state is retained until assembly or reset.	**Implemented**

5. Relation to the planned scope

The project scope document lists the web simulator (M1), instruction encoder/decoder (M5), and memory explorer (M7) as must-have goals. These are the current achieved areas. The scope also expects integration across modules and validation against known results; these broader integration and verification tasks remain in progress.

Scope area	Current status	Notes
Web simulator	**Achieved**	Connected web workflow built on the Kites engine.
Memory explorer	**Achieved**	Paged memory inspection and byte-level interaction.
Instruction encoding/decoding	**Achieved for supported formats**	Uses the existing assembler and instruction representation.
Compiler comparison / cross compilation	Pending	Not part of the current delivered browser workflow; planned as a new page.
Processor design studio / custom ISA	Pending	Present in the broader scope; not included in the current implementation.
Cache and branch prediction labs	Pending	Architecture analysis work is planned next.
Integrated verification and final report	Pending / ongoing	Cross-module tests, documented metrics, and final demonstration remain to be completed.

6. Changes from the planned scope

No change to the project goals is being reported. Work has been delivered in phases, starting with a web front end connected to the existing C++ simulator and assembler. This uses the established Kites engine as the source of execution truth and gives the team a working platform to extend. Items in the scope document that are not yet implemented are deferred work, rather than removed goals.

Pending work and next phase

The next work extends the current simulator foundation toward the compiler and architecture-analysis parts of the approved scope.

7. Pending tasks

Cross compilation page

Create a browser page for selecting supported compiler/target options, compiling source, and presenting compiler output for comparison with assembly and machine code. Define the available compiler/toolchain boundary and show clear diagnostics when a toolchain is unavailable.

Branch prediction lab

Build a trace-driven lab that shows predicted and actual branch outcomes and reports the predictor behavior and metrics specified by the scope.

Cache lab

Build a trace-driven cache lab with supported mapping/associativity choices and visible hits, misses, evictions, and rates.

Integration and validation

Connect new modules through shared program/trace data, verify results against hand-checked or reference cases, and document supported configurations and limitations.

Remaining scope work

Plan the custom ISA and processor design studio components from the original scope, then complete end-to-end documentation and the final demonstration.

8. Planned next development

Next	Work item	Expected outcome
1	Cross compilation page	A clear browser workflow for compiling and comparing source, assembly, and machine output.
2	Branch prediction lab	An interactive analysis view for prediction outcomes and accuracy metrics.
3	Cache lab	An interactive cache analysis view for accesses, hits, misses, evictions, and rates.

Immediate planned work: build the cross compilation page, then implement the branch prediction and cache lab.

Project status is reported as of 09 November 2026. Features described as implemented refer to the current project state; pending items remain planned development.